

LinkedList Interview Handbook

Professional revision notes in the same style we used while learning.

WHY WAS LINKEDLIST CREATED?

ArrayList is excellent for reading data quickly, but inserting or deleting in the middle is expensive because elements must shift. LinkedList was designed for situations where modifications happen frequently.

REAL-LIFE ANALOGY

Imagine a train:

Engine → Coach1 ↔ Coach2 ↔ Coach3 ↔ Coach4

Every coach knows who is before it and who is after it. A LinkedList node works the same way.

INTERNAL WORKING

Each node stores:

- Previous node
- Data
- Next node

Node Structure:

[Prev] ↔ [Data] ↔ [Next]

WHY get(index) IS SLOW

Nodes are not stored continuously like an array. Java must travel node by node until it reaches the requested index, therefore get(index) is $O(n)$.

WHY INSERT/REMOVE IS FAST

Before:

10 ↔ 20 ↔ 30

Remove 20:

10 ↔ 30

No shifting occurs. Java only changes the links between nodes.

TIME COMPLEXITY

Operation	Complexity	Reason
get(index)	$O(n)$	Traversal

addFirst/addLast	O(1)	Head/Tail update
removeFirst/removeLast	O(1)	Head/Tail update
contains	O(n)	Sequential traversal

INTERVIEW TIP

Choose ArrayList for frequent random access. Choose LinkedList when insertions/removals happen frequently, especially at the beginning or end.

COMMON MISTAKE

- LinkedList is not always faster than ArrayList.
- Fast insertion doesn't mean fast search.
- Reaching the insertion position still requires traversal.

REMEMBER FOREVER

LinkedList = Doubly Linked List
Fast insert/remove.
Slow random access.
Best for frequent modifications.